第二节 Neo4j 基本使用

安装好 Neo4j 后。我们就可以学习一些 Neo4j 的基本用法了。

一、创建并连接数据库

在使用 Neo4j 进行开发时，首先需要在 Neo4j Desktop 中创建一个本地数据库实例（Instance）。这个过程非常直观。

1. **创建实例**：打开 Neo4j Desktop，在 “Local instances” 页面点击 “Create instance” 按钮。

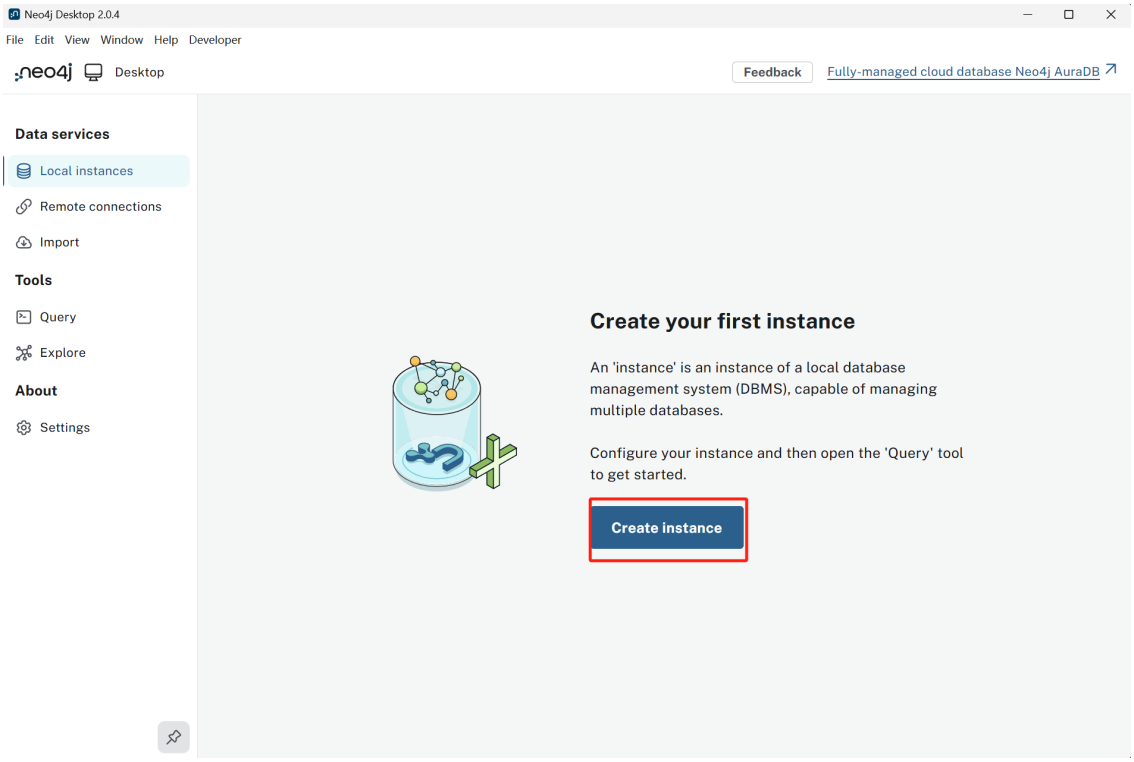

图 2.1: 点击创建实例

2. **配置实例**：在弹出的窗口中，为实例命名（例如 `base nlp`），选择所需的 Neo4j 版本，并为默认用户 `neo4j` 设置一个能记住的密码。完成后点击 “Create”。

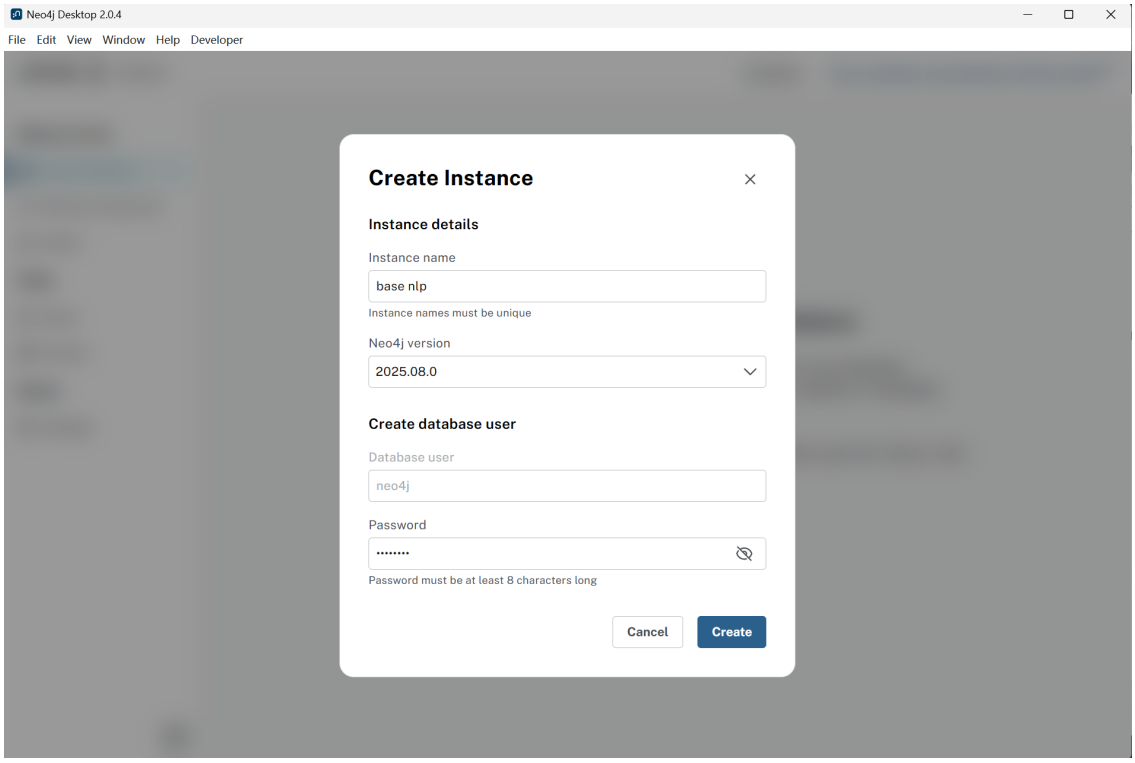

图 2.2: 配置实例信息

3. **启动与连接**：实例创建后会自动启动，状态显示为 “RUNNING”。此时，可以通过浏览器直接访问 `http://127.0.0.1:7474` 来打开 Neo4j Browser。

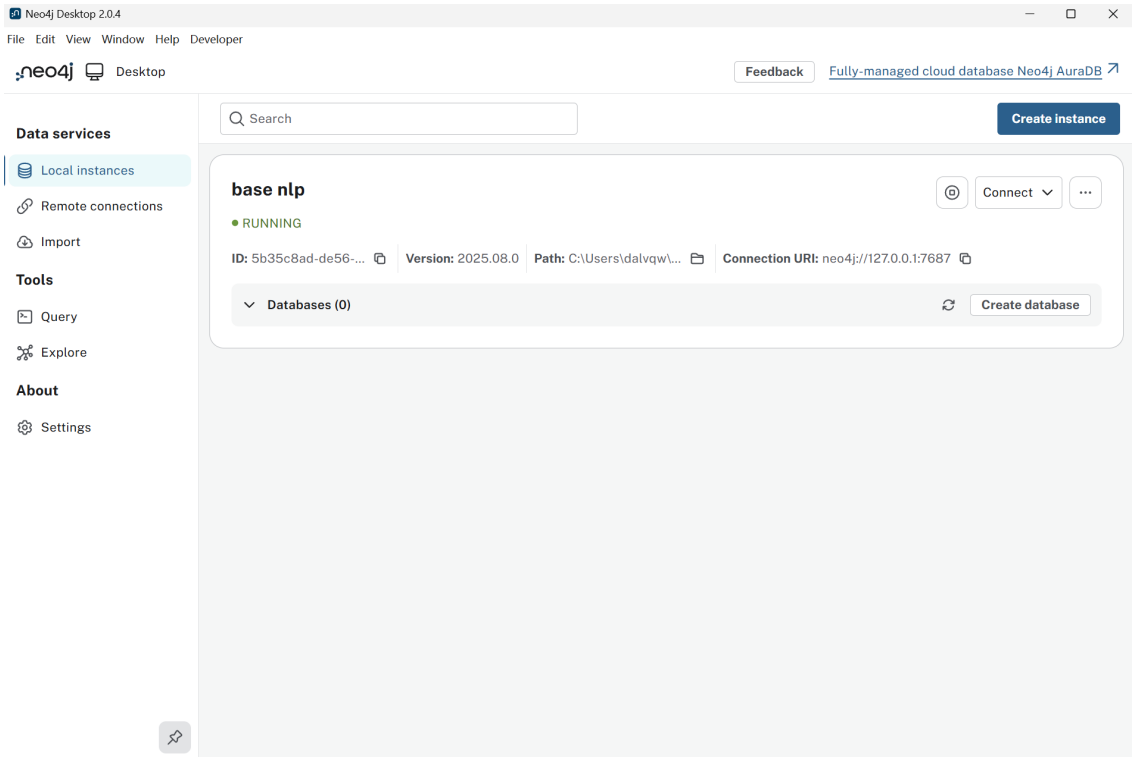

图 2.3: 实例创建成功并运行

在浏览器打开的连接界面中，使用刚刚设置的密码进行连接。

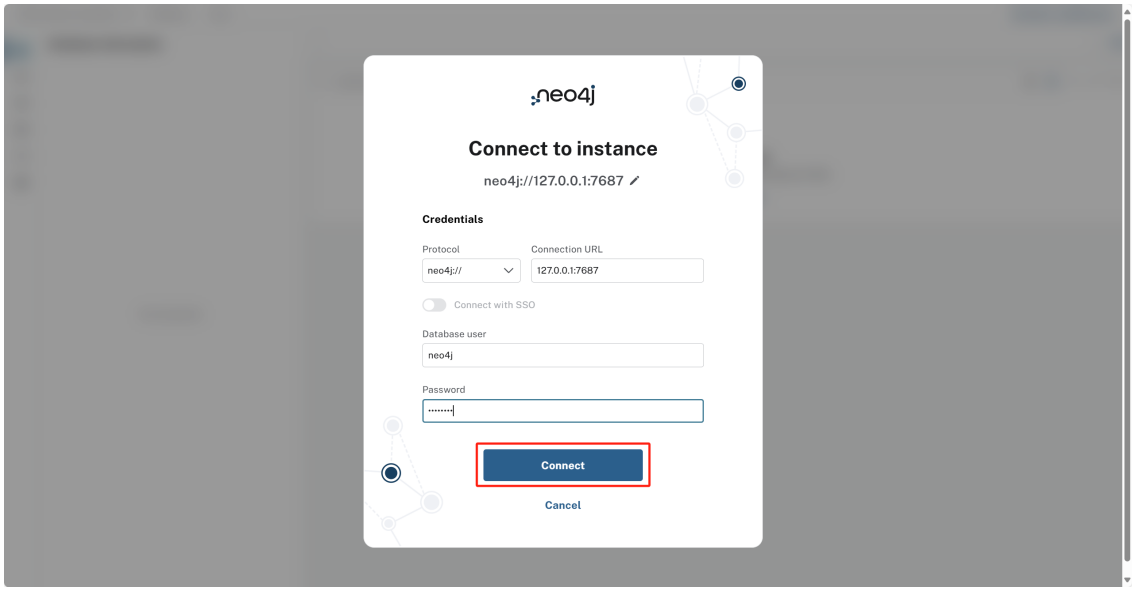

图 2.4: 使用密码连接数据库

二、增删查改

数据库操作的核心无外乎增删查改（CRUD），下面来使用 Cypher，围绕一个菜品信息图谱的场景，逐一介绍这些基本操作。

2.1 场景设定

为了方便演示，先设定好本次实践所需要用到的实体、属性和关系。

- **实体/标签 (Labels)**:
  - `Ingredient`: 食材，拥有 `name`, `category`（类别），`origin`（产地），`tags`（标签，数组）等属性。
  - `Dish`: 菜品，拥有 `name`, `cuisine`（菜系）等属性。
- **关系 (Relationships)**:
  - `(Dish)-[:包含]->(Ingredient)`: 表示某菜品包含某种食材，关系上可以有 `用量` 属性。
  - `(Dish)-[:主要食材]->(Ingredient)`: 表示某菜品的主要食材是某种食材。
  - `(Dish)-[:调味]->(Ingredient)`: 表示某菜品使用某种食材进行调味。

2.2 创建 (CREATE)

`CREATE` 语句用于在图中创建新的节点和关系。

2.2.1 创建节点

创建节点的基本语法是 `CREATE (变量:标签 {属性: 值})`。

- **变量 (Variable)**: 如 `pork`，是一个临时名称，用于在同一条语句中引用该节点。如果后续不需要引用，可以省略。
- **标签 (Label)**: 如 `Ingredient`，用于对节点进行分类。
- **属性 (Properties)**: 一个包含键值对的 map/字典，用于描述节点的具体信息。

最基础的创建语句包含一个临时变量（`pork`）、一个标签（`Ingredient`）和一组属性。

```
CREATE (pork:Ingredient {name:'猪肉', category:'肉类', origin:'杭州'});
```

如果在创建后不需要立刻使用这个节点（例如，在同一查询中创建关系），可以省略临时变量名，这样语法更简洁。

```
CREATE (:Ingredient {name:'土豆', category:'蔬菜', origin:'北京'});
```

还可以在创建节点后，使用 `RETURN` 子句立即将其返回。这对于调试或确认节点是否按预期创建非常有用。`RETURN n` 会在结果面板中直接显示刚刚创建的 `鸡蛋` 节点的信息。

```
CREATE (n:Ingredient {name:'鸡蛋'}) RETURN n;
```

执行上述三条命令后，数据库中就创建了三个 `Ingredient` 类型的节点。能够通过 Neo4j Browser 的可视化界面直观地看到这些新创建的数据。

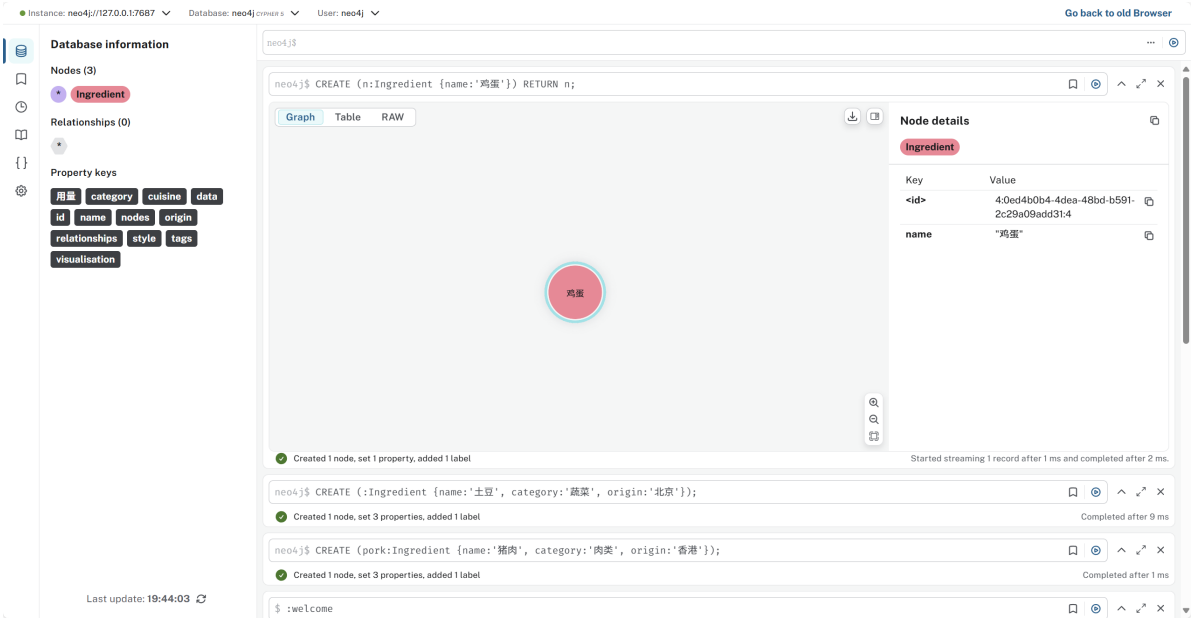

图 2.5: 执行创建命令后，左侧面板显示已有 3 个 Ingredient 节点

点击左侧面板中的 `Ingredient` 标签，Neo4j Browser 会自动执行 `MATCH (n:Ingredient) RETURN n LIMIT 25;` 查询，并在主窗口中展示所有食材节点。如图 2.6 所示，点击其中一个节点（如“土豆”），右侧会显示其详细属性。这里可以观察到：

- `<id>` **字段**：这是 Neo4j 为每个节点自动生成的内部唯一标识符。
- **Key-Value 结构**：右侧的 “Key” 和 “Value” 两列展示了节点属性是以键值对的形式存储的。
- **自定义属性**：`name`、`category`、`origin` 三个字段的值与前面 `CREATE` 语句中设定的值完全一致。

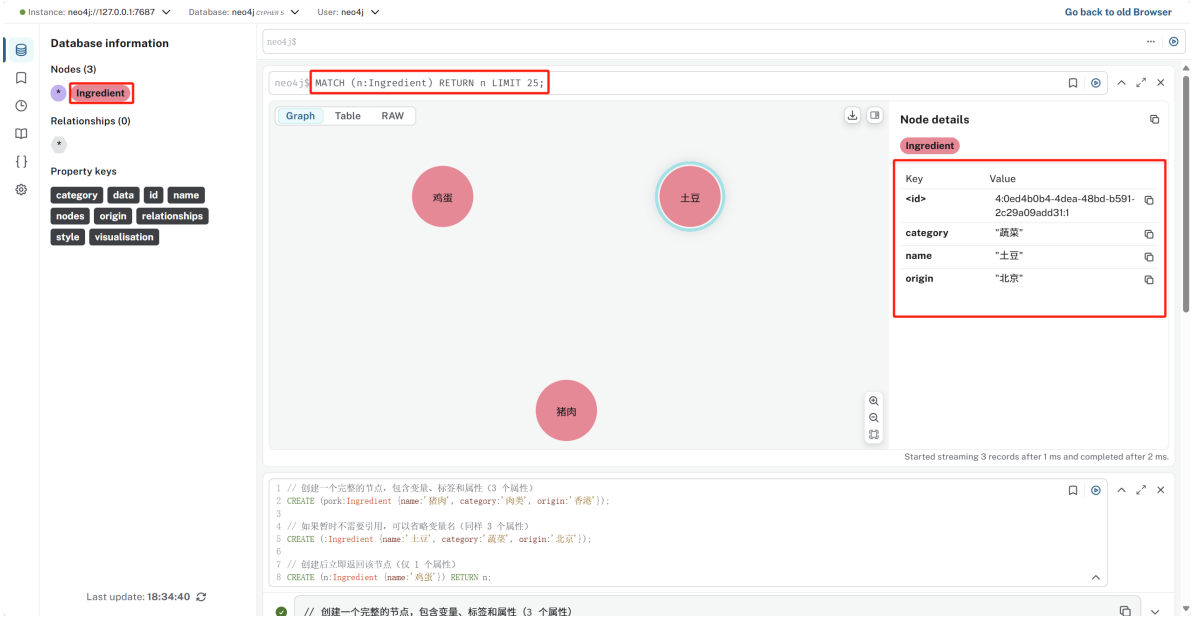

图 2.6: 查询并查看新创建的节点及其属性

2.2.2 创建关系

关系的创建通常需要先指定关系两端的节点，然后用 `-[变量:类型 {属性}]->` 来定义关系。

- 关系必须有 **方向** 和 **类型 (Type)**。
- 小括号 `()` 用于表示节点，中括号 `[]` 用于表示关系。

在实际应用中，常常需要一次性创建多个节点以及它们之间的关系。`CREATE` 语句支持通过逗号分隔，在一个查询中完成复杂图谱的构建。下面的例子将创建一个更复杂的菜品关系网络，以体现“多对多”的特性（一道菜包含多种食材，一种食材可用于多道菜）。

```
CREATE
    // 创建食材节点
    (rousi:Ingredient {name:'猪里脊'}),
    (muer:Ingredient {name:'木耳'}),
    (huluobo:Ingredient {name:'胡萝卜'}),
    (qingjiao:Ingredient {name:'青椒'}),
    // 创建菜品节点
    (d1:Dish {name:'鱼香肉丝', cuisine:'川菜'}),
    (d2:Dish {name:'木须肉', cuisine:'鲁菜'}),
    // 创建关系
    (d1)-[:包含 {amount:'250g'}]->(rousi), (d1)-[:包含]->(muer), (d1)-[:包含]->
(huluobo),
    (d2)-[:包含 {amount:'150g'}]->(rousi), (d2)-[:包含]->(muer),
    // 创建双向关系
    (rousi)-[:被用于]->(d1), (muer)-[:被用于]->(d1), (huluobo)-[:被用于]->(d1),
    (rousi)-[:被用于]->(d2), (muer)-[:被用于]->(d2);
```

这个查询语句做了以下几件事：

1. **创建了 4 个** `Ingredient` **节点**：猪里脊、木耳、胡萝卜、青椒。
2. **创建了 2 个** `Dish` **节点**：鱼香肉丝、木须肉。
3. **创建了 5 条** `包含` **关系**：从菜品指向食材。
4. **创建了 5 条** `被用于` **关系**：从食材指向菜品。这样既可以方便地查询“一道菜包含哪些食材”，也可以高效地反向查询“一种食材被用在了哪些菜里”。

执行 `MATCH p=()-[:包含]->() RETURN p LIMIT 25;` 查询可以可视化展示所有“包含”关系。点击关系（箭头），可以在右侧看到其详细信息，例如“鱼香肉丝”到“猪里脊”的关系上，就包含了在 `CREATE` 语句中定义的 `amount: '250g'` 这一属性。

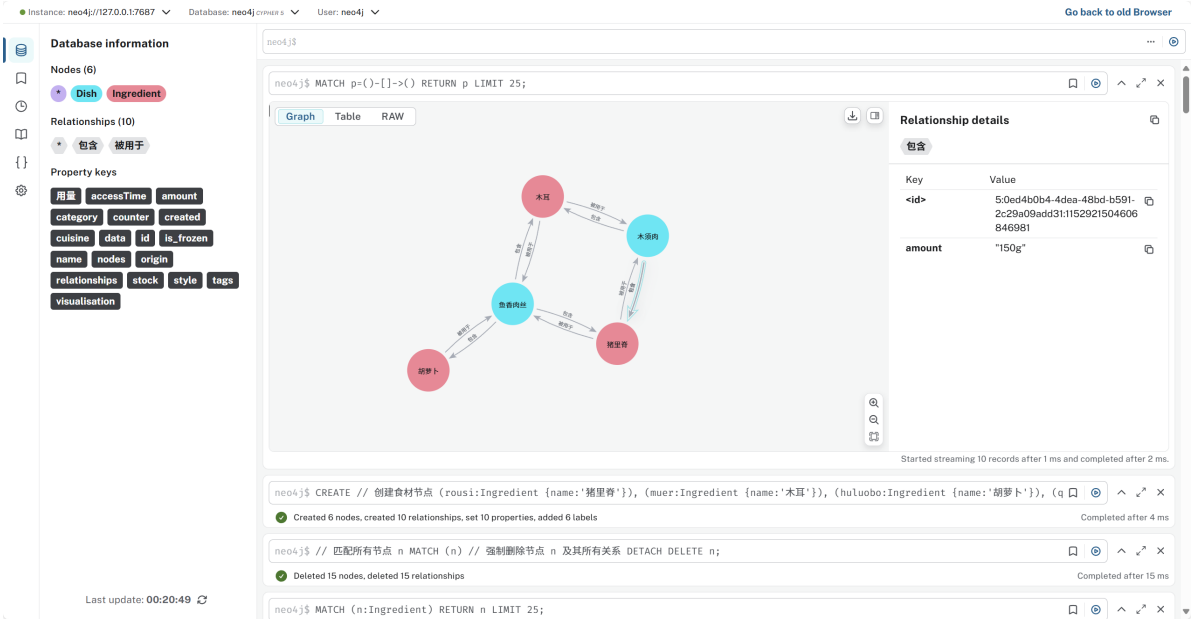

图 2.7: 创建关系后的图谱结构

2.3 查询 (MATCH)

`MATCH` 是 Cypher 中用于查询图数据的命令，它允许你描述你想要寻找的节点和关系的模式。

2.3.1 基本查询

最简单的查询是匹配并返回图中的任意节点，可以使用 `LIMIT` 关键字限制返回数量，避免因数据量过大导致浏览器卡顿。

```
// 匹配并返回图中的任意 25 个节点
MATCH (n)
RETURN n
LIMIT 25;
```

也可以根据标签和属性进行精确匹配。

```
// 匹配所有标签为 Ingredient，且名字为'猪里脊'的节点
MATCH (n:Ingredient {name:'猪里脊'}) RETURN n;
```

2.3.2 条件查询 (WHERE)

`WHERE` 子句提供了更灵活的过滤能力，可以对节点的属性进行复杂的逻辑判断。

例如，查询名字是'猪里脊'或'鸡蛋'的 `Ingredient` 节点。

```
MATCH (n:Ingredient)
WHERE n.name IN ['猪里脊','鸡蛋']
RETURN n;
```

也可以使用 `AND`、`OR` 等关键字构建复合查询条件。

```
// 复合条件：查询指定名称且类别为“肉类”的节点
MATCH (n:Ingredient)
WHERE n.name IN ['猪肉', '猪里脊', '鸡蛋'] AND n.category = '肉类'
RETURN n;
```

2.3.3 返回指定属性

默认情况下，`RETURN n` 会返回整个节点对象。也可以只返回节点的特定属性，并使用 `AS` 为返回的列起别名，使结果更具可读性。

```
MATCH (n:Ingredient)
WHERE n.name IN ['猪里脊','鸡蛋']
RETURN n.name AS 食材名称, n.category AS 类别;
```

2.3.4 关联查询

图数据库最强大的地方在于对关系的查询。例如，可以一次性查询“鱼香肉丝”和“木须肉”分别包含了哪些食材。

```
MATCH (d:Dish)-[:包含]->(i:Ingredient)
WHERE d.name IN ['鱼香肉丝', '木须肉']
RETURN d.name AS 菜品, collect(i.name) AS 食材列表;
```

> `collect()` 是一个聚合函数，可以将匹配到的多个同类结果（这里是食材名称 `i.name`）收集到一个列表中。

2.3.5 查询并创建 (MATCH + CREATE)

在实际应用中，一个常见的操作是先找到图中已经存在的节点，然后为它们添加新的关系。这可以通过组合使用 `MATCH` 和 `CREATE` 来实现。

例如，我们已经创建了“鱼香肉丝”和“猪里脊”，现在想为它们添加一条“主要食材”的关系。

```
MATCH
    (d:Dish {name:'鱼香肉丝'}),
    (i:Ingredient {name:'猪里脊'})
MERGE
    (d)-[r:主要食材]->(i)
RETURN d, i, r;
```

> 这个模式确保了是在已有的、正确的实体之间建立关联，并通过 `MERGE` 避免重复的关系。

2.3.6 排序 (ORDER BY)

可以使用 `ORDER BY` 子句对返回的结果进行排序。默认是升序（`ASC`），也可以指定为降序（`DESC`）。

```
// 查询所有食材，并按名称升序排序
MATCH (i:Ingredient)
RETURN i.name, i.category
ORDER BY i.name ASC;
```

2.4 更新 (SET & MERGE)

2.4.1 更新属性 (SET)

`SET` 语句用于修改或添加节点/关系的属性。它必须和 `MATCH` 配合使用，先找到要更新的实体，再进行修改。

```
MATCH (i:Ingredient {name:'猪肉'})
SET
    i.is_frozen = true,
    i.origin = '金华'
RETURN i;
```

2.4.2 插入或更新 (MERGE)

在构建知识图谱时，经常遇到这样的场景：如果某个节点已存在，则更新其属性；如果不存在，则创建它。`MERGE` 语句就可以解决这个问题。

`MERGE` 会根据你提供的模式在图中查找，如果找到匹配项，则执行 `ON MATCH` 部分；如果未找到，则执行 `ON CREATE` 部分，从而避免了重复创建实体。

```
// 查找名为'大蒜'的 Ingredient 节点
MERGE (n:Ingredient {name: '大蒜'})
// 如果不存在，则创建该节点，并设置创建时间和初始库存
ON CREATE SET
    n.created = timestamp(),
    n.stock = 100
// 如果已存在，则更新其库存、访问次数和访问时间
ON MATCH SET
  n.stock = coalesce(n.stock, 0) - 1,
  n.counter = coalesce(n.counter, 0) + 1,
  n.accessTime = timestamp()
RETURN n;
```

> `coalesce(property, defaultValue)` 是一个非常有用的函数，它会检查属性 `property` 是否存在，如果存在则返回其值，否则返回 `defaultValue`。

2.5 删除 (DELETE & REMOVE)

2.5.1 删除属性 (REMOVE)

`REMOVE` 用于移除节点或关系上的某个属性。在下面的例子中，先用 `MATCH` 找到名为“大蒜”的节点，然后移除由 `MERGE` 命令在创建它时添加的 `created` 属性。

```
MATCH (i:Ingredient {name:'大蒜'})
REMOVE i.created
RETURN i;
```

2.5.2 删除节点和关系 (DELETE)

`DELETE` 用于删除节点和关系。但需要 **特别注意**：Neo4j 不允许直接删除一个还存在关联关系的节点。你必须先删除关系，才能删除节点。

```
// 错误示范：如果'大蒜'还有关系连着，这条语句会报错
MATCH (i:Ingredient {name:'大蒜'})
DELETE i;
```

正确的做法有两种。第一种是先手动删除与节点相关的所有关系，然后再删除节点本身。

```
// 正确做法 1：先删除关系，再删除节点
MATCH (i:Ingredient {name:'大蒜'})-[r]-() // 匹配与'大蒜'相连的任意关系
DELETE r, i; // 先删除关系 r，再删除节点 i
```

第二种做法更简洁，也是官方推荐的方式：使用 `DETACH DELETE`。它会自动删除指定节点以及所有与它直接相连的关系。

```
// 正确做法 2：使用 DETACH DELETE (推荐)
MATCH (i:Ingredient {name:'大蒜'})
DETACH DELETE i;
```

此外，还可以通过节点的内部 ID 进行精确查找和删除。每个节点都有一个由 Neo4j 自动分配的唯一 ID，可以通过 `id()` 函数获取。

```
// 假设我们通过查询得知“大蒜”的 ID 为 5
MATCH (i:Ingredient)
WHERE id(i) = 5
DETACH DELETE i;
```

2.5.3 清空数据库

如果想删除数据库中的所有节点和关系，可以使用以下命令：

```
// 匹配所有节点 n
MATCH (n)
// 强制删除节点 n 及其所有关系
DETACH DELETE n;
```

2.5.4 软删除

在生产环境中，直接从数据库中物理删除（`DELETE`）数据是一种高风险操作。一种更安全、更常见的做法是“软删除”。软删除并非真的将数据移除，而是通过 `SET` 命令为其添加一个状态属性，将其标记为“已删除”或“不活跃”。

```
// 将“木耳”标记为不活跃
MATCH (i:Ingredient {name:'木耳'})
SET i.is_active = false;
```

这样，在后续的查询中，只需要增加一个 `WHERE i.is_active = true` 的过滤条件，就能只使用那些“活跃”的数据，而被软删除的数据依然保留在数据库中，以备审计或恢复。

> 删除操作是高风险行为，尤其是在生产环境中。执行前请务必确认操作对象和范围，并做好数据备份。

三、批量导入